

UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 8 – MACHINE TRANSLATION APPLICATION

English Engineering Document to Hindi

Professional Implementation Report

Question	Develop a machine translation application that translates an engineering document from English into another Indian language using a pre-trained translation model.
Target Language	Hindi
Folder Name	Q8_Machine_Translation
Python File	machine_translation.py
AI Service	Hugging Face Inference API with a pre-trained English-to-Hindi translation model

1. AIM

To develop a machine translation application that accepts an English engineering document and translates its content into Hindi using a pre-trained machine translation model.

2. OBJECTIVES

- Accept an English engineering PDF document.
- Extract readable text from the uploaded PDF.
- Divide lengthy text into manageable chunks for translation.
- Translate each chunk from English into Hindi using a pre-trained translation model.
- Combine the translated chunks into a complete Hindi version.
- Display both the English source and Hindi translation in an interactive interface.
- Provide an option to download the translated text.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- PyPDF
- Hugging Face Hub / Inference API
- Pre-trained English-to-Hindi translation model
- Internet connection
- Computer or laptop

4. FOLDER AND FILE STRUCTURE

```
UNIT_IV_Multimodal_Generative_AI
├── Q8_Machine_Translation
│   └── machine_translation.py
```

5. TECHNOLOGIES USED

Technology	Purpose	Role in Application
Python	Programming	Implements the translation workflow
Streamlit	Web interface	Provides PDF upload and translation display
PyPDF	PDF processing	Extracts English text from the document
Hugging Face Inference API	AI inference	Provides access to the translation model
Pre-trained Translation Model	Language translation	Translates English engineering text into Hindi

6. SYSTEM WORKFLOW

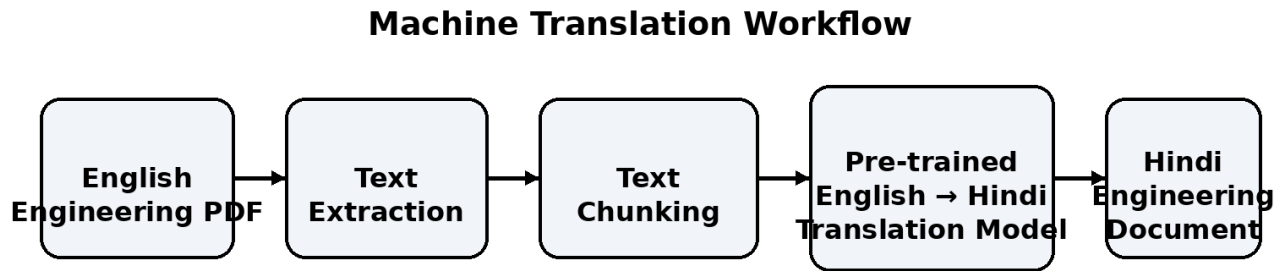


Figure 1. English-to-Hindi machine translation workflow.

The application receives an English engineering PDF, extracts the text, divides it into manageable sections, sends each section to the pre-trained English-to-Hindi translation model, and combines the translated sections into a Hindi engineering document.

7. STEP-BY-STEP IMPLEMENTATION

Step 1: Create the Q8_Machine_Translation folder inside UNIT_IV_Multimodal_Generative_AI.

Step 2: Create the machine_translation.py Python file.

Step 3: Install Streamlit, Hugging Face Hub and PyPDF.

Step 4: Configure the Hugging Face access token using the HF_TOKEN environment variable.

Step 5: Initialize the Hugging Face InferenceClient.

Step 6: Create a PDF file uploader in Streamlit.

Step 7: Extract text from all PDF pages using PyPDF.

Step 8: Divide the extracted text into manageable chunks.

Step 9: Send each English chunk to the pre-trained English-to-Hindi translation model.

Step 10: Collect and combine all translated chunks.

Step 11: Display the English source and Hindi translated document.

Step 12: Provide a download button for the Hindi translation.

8. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st
from huggingface_hub import InferenceClient
from pypdf import PdfReader
import os

st.set_page_config(
    page_title="Engineering Document Translator",
    page_icon="🌐",
```

```

        layout="wide"
    )

st.title("🌐 Engineering Document Translator")

st.write(
    "Translate an English engineering document into Hindi "
    "using a pre-trained machine translation model."
)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:
    st.error("HF_TOKEN is not configured.")
    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

uploaded_file = st.file_uploader(
    "Upload a PDF document",
    type=["pdf"]
)

def extract_text(pdf_file):
    reader = PdfReader(pdf_file)
    pages = []

    for page in reader.pages:
        text = page.extract_text()
        if text:
            pages.append(text.strip())

    return "\n".join(pages)

def split_text(text, max_chars=1800):
    words = text.split()
    chunks = []
    current = []
    length = 0

    for word in words:
        if length + len(word) + 1 > max_chars:
            if current:
                chunks.append(" ".join(current))
                current = []
                length = 0

        current.append(word)
        length += len(word) + 1

    if current:
        chunks.append(" ".join(current))

```

```

return chunks

def translate_chunk(text):
    result = client.translation(
        text=text,
        model="Helsinki-NLP/opus-mt-en-hi"
    )
    return result.translation_text

def translate_document(text):
    chunks = split_text(text, max_chars=1800)
    translations = []

    for chunk in chunks:
        translated = translate_chunk(chunk)

        if translated.strip():
            translations.append(translated.strip())

    return "\n\n".join(translations)

if uploaded_file is not None:

    st.success(f"Uploaded: {uploaded_file.name}")

    if st.button("🌐 Translate to Hindi"):

        try:
            with st.spinner(
                "Extracting English engineering document..."
            ):
                english_text = extract_text(uploaded_file)

            if not english_text.strip():
                st.warning(
                    "No readable text was found in the PDF."
                )
                st.stop()

            word_count = len(english_text.split())

            st.info(
                f"Extracted approximately "
                f"{word_count:,} English words."
            )

            with st.spinner(
                "Translating engineering document..."
            ):
                hindi_text = translate_document(
                    english_text

```

```

    )

    st.subheader("GB English Source")

    st.text_area(
        "English Document Text",
        english_text,
        height=250
    )

    st.subheader("IN Hindi Translation")

    st.text_area(
        "Translated Engineering Document",
        hindi_text,
        height=350
    )

    st.download_button(
        label="⬇ Download Hindi Translation",
        data=hindi_text,
        file_name="engineering_document_hindi.txt",
        mime="text/plain"
    )

    st.success(
        "Engineering document translated successfully."
    )

except Exception as e:
    st.error("Machine translation failed.")
    st.code(str(e))

```

9. EXECUTION COMMAND

```
streamlit run ".\Q8_Machine_Translation\machine_translation.py"
```

10. TEST DOCUMENT

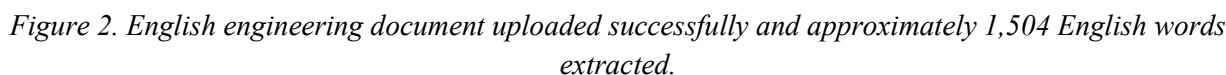
The application was tested using the engineering technical document “Engineering_Operating_Systems_Technical_Document.pdf”. The document contains operating-system concepts written in English and is suitable for demonstrating engineering document translation.

11. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show the actual successful execution of the application, including document upload, English text extraction, and the generated Hindi translation.

Translate an English engineering document into Hindi using a pre-trained machine translation model.

Upload a PDF document



English Document Text

IN Hindi Translation 

Translated Engineering Document

Figure 3. Generated Hindi translation of the engineering document.

Test Case	Input	Observed Result
1	Engineering Operating Systems PDF	PDF uploaded and 1,504 English words extracted.
2	English engineering text	Translated into Hindi successfully.
3	Full document translation	Hindi translation displayed in the Streamlit text area.

- The PDF was successfully uploaded and processed.
- The application extracted approximately 1,504 English words from the engineering document.

- The document was translated into Hindi using a pre-trained translation model.
- The translated output was displayed clearly in the application.
- The translation can be downloaded as a text file.

14. RESULT

The Engineering Document Translator was successfully implemented. The application accepted the English engineering document, extracted approximately 1,504 words, translated the content into Hindi using a pre-trained machine translation model, and displayed the translated engineering document through the Streamlit interface.

15. ADVANTAGES

- Automatically translates engineering documents.
- Uses a pre-trained translation model without training from scratch.
- Supports long documents through text chunking.
- Provides a simple and interactive web interface.
- Can be extended to other Indian languages.

16. LIMITATIONS

- Technical terms may sometimes be translated inconsistently.
- Translation quality depends on the selected pre-trained model.
- Internet access is required for API-based translation.
- Very large documents may require multiple translation requests.

17. FUTURE ENHANCEMENTS

- Support Tamil, Telugu, Kannada, Malayalam and other Indian languages.
- Provide a language-selection dropdown.
- Preserve PDF formatting and generate a translated PDF.
- Add translation quality evaluation and terminology consistency checks.
- Support DOCX and TXT engineering documents.

18. CONCLUSION

The developed machine translation application demonstrates a practical use of Generative AI and NLP for engineering education. By combining PDF text extraction, chunk-based processing and a pre-trained English-to-Hindi translation model, the application can convert an engineering document into another Indian language. The successful test results confirm that the required document translation functionality has been achieved.

19. VIVA / KEY POINTS

- Machine translation automatically converts text from one language into another.
- A pre-trained translation model is used instead of training a translation system from scratch.
- PyPDF extracts the source text from the engineering PDF.
- Chunking allows large documents to be translated in manageable sections.

- The same architecture can be extended to other Indian languages.